U.PORTO

FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

8-Ball Pool
Ball Detection

Task 1

Computer Vision 2025/2026

Authors:
António Santos (up202205469@up.pt) - T01
Luís Arruda (up202206970@up.pt) - T01
Vanessa Queirós (up202207919@up.pt) - T01

1. Introduction

This project implements a computer vision pipeline to detect and identify pool balls on an 8-ball table. Using OpenCV and NumPy, the system locates balls and determines their numbers through color analysis and geometric detection in a modular workflow.

2. Methodology

2.1 Top-View of the Table

1. Color Segmentation

The image is converted to the HSV color space, which is more robust to variations in lighting, and a filter is applied that isolates only the pixels within a previously defined blue range. The largest continuous blue blob with an area above a minimum threshold is selected as the table region.

2. Quadrilateral Estimation

The convex hull is calculated over the outline. The *'select_table_quad'* function attempts, in order of preference:

1. Fit a line to each side via *'cv2.fitLine'* and calculate the four intersections, validating whether the internal angles are close to 90°. This verification therefore serves as a quality gate: if the quadrilateral estimated by this method has internal angles that deviate too much from 90°, the result is discarded and the pipeline reverts to a simpler and more conservative method.
2. Approximate the hull to a 4-vertex polygon using *approxPolyDP*;
3. Or use the minimum bounding rectangle as a fallback.

3. Adaptive Padding and Homography Calculation

A margin is added around the rectified region, proportional to the asymmetry between opposite sides, to ensure that no part of the table is cut off. The four identified corners are ordered clockwise, and the actual dimensions of the table are estimated by comparing the lengths of the opposite sides, with an amplification factor that corrects for perspective distortion. Based on this, a geometric transformation is calculated that projects the table into a strictly frontal view.

2.2 Ball Detection

For ball detection we have 3 main steps to try to correctly identify the positions and the center of each ball. The first part starts by defining the pool table length and the pool ball diameter. This is important because since every image contains a different perspective the diameter of the ball changes from photo to photo. To help with this the ball detection uses a function which given the original image finds the four dots of the warped table and orders them so that we can then create a rectangle which allows us to get an approximation of the pool table size in pixels.

Following this we convert the real values to a ratio (POOL_TABLE_BALL_RATIO = POOL_BALL_DIAMETER / POOL_TABLE_LENGTH) which is then used to approximate the size of a ball in pixels. The algorithm then goes on to first eliminate every part of the image that is not relevant to image detection by using a mask such that only the balls and the blue part of the table is visible.

The next part of the algorithm is the use of HoughCircles which was optimized by using Optuna such that the results were as close as possible to the example which was given. There was also some trial and error to finalize some of the parameters that HoughCircles uses to correct some mistakes on the rest of the images. Finally the function comes to an end by using a *for loop* to iterate through the found circles. In this final *for loop* we check every ball to try to eliminate possible outliers (e.g. checking if the ball color is similar to the table color).

2.3 Number Identification

After detecting the ball positions, each ball is analysed individually to determine its type and number. The detected circles are first cropped into Regions of Interest (ROIs), which isolate each ball from the original image. To improve robustness under different lighting conditions, a white balance calibration step is applied using the cue ball as a reference. The cue ball is identified as the brightest ball with the lowest saturation, and this information is used to adjust the brightness of the entire image for more consistent color detection.

Each ball ROI is converted to the HSV color space, which separates color information from brightness, making color classification more reliable. A circular mask is applied to focus only on the ball surface, and the dominant color is computed using the median of masked pixels. The system also checks whether the ball is striped by analysing the percentage of white pixels across the ball.

Finally, the detected color is mapped to a corresponding pool ball number using predefined HSV hue ranges. Solid balls are assigned numbers 1-7, striped balls are assigned numbers 9-15, the cue ball is labelled as 0, and the black ball is detected using low brightness values. The result is a structured list of detected balls, each containing its number and bounding box coordinates.

3. Results

The results indicate that the system performs reasonably well at detecting ball locations, with good alignment between predicted and actual positions. This reflects the effectiveness of the detection stage shown in *Fig. 2*. However, some balls remain undetected, suggesting that challenges such as lighting conditions and occlusions still affect performance, despite the table masking approach illustrated in *Fig. 1*.

Classification performance is more limited, with a significant portion of detected balls being incorrectly identified. This highlights difficulties in distinguishing colours and ball types, as demonstrated in Fig. 3. Overall, while detection is relatively reliable, improving ball identification proved very difficult and nearly impossible to refine further due to varying angles and lighting conditions. A potential improvement would be the use of convolutional neural networks (CNNs) for more robust classification.

References

1. OpenCV Documentation. Open Source Computer Vision Library. Available at: https://docs.opencv.org/4.x/
2. Optuna Contributors. Optuna: A Next-generation Hyperparameter Optimization Framework, 2019. Available at: https://optuna.org/
3. NumPy Contributors. NumPy Documentation, 2024. Available at: https://numpy.org/doc/
4. dijdomv01. A Beginner's Guide to Understand the Color Models RGB and HSV, 2024. Available at: https://medium.com/@dijdomv01/a-beginners-guide-to-understand-the-color-models-rgb-and-hsv-244226e4b3e3
5. elvenkim1. Understanding Hough Circles for Holes Detection, 2024. Available at: https://medium.com/@elvenkim1/understanding-hough-circles-for-holes-detection-5b686d97aa04

Appendix

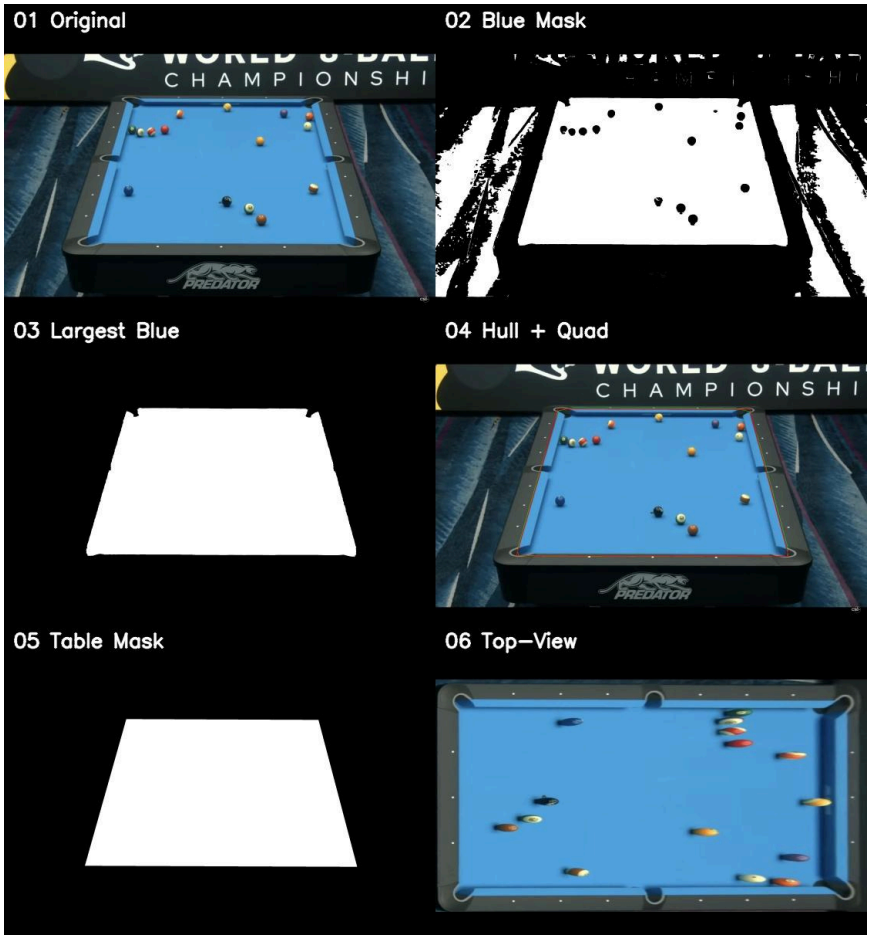


Fig. 1 - Computations of Table Mask

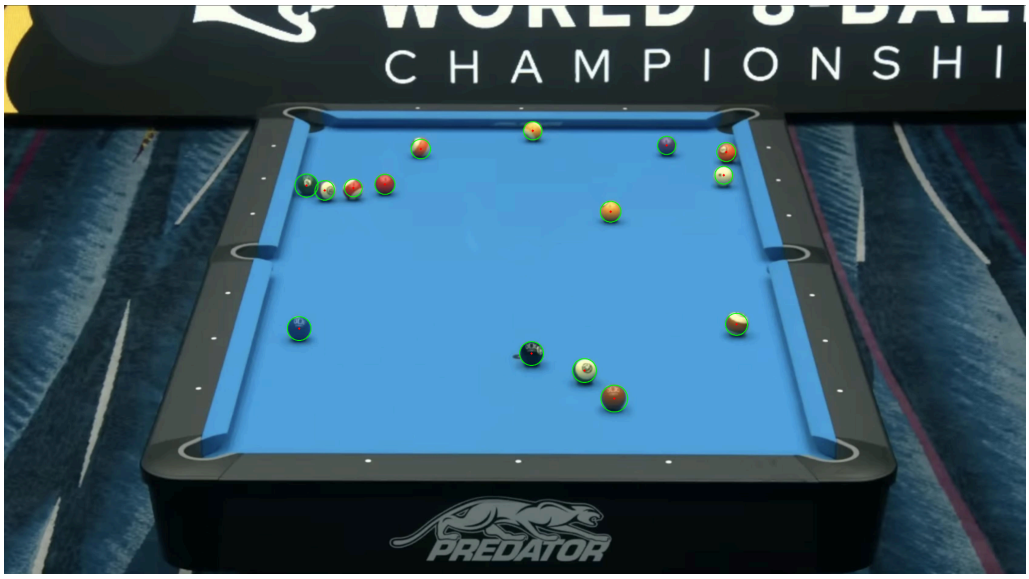


Fig. 2 - Ball Detection

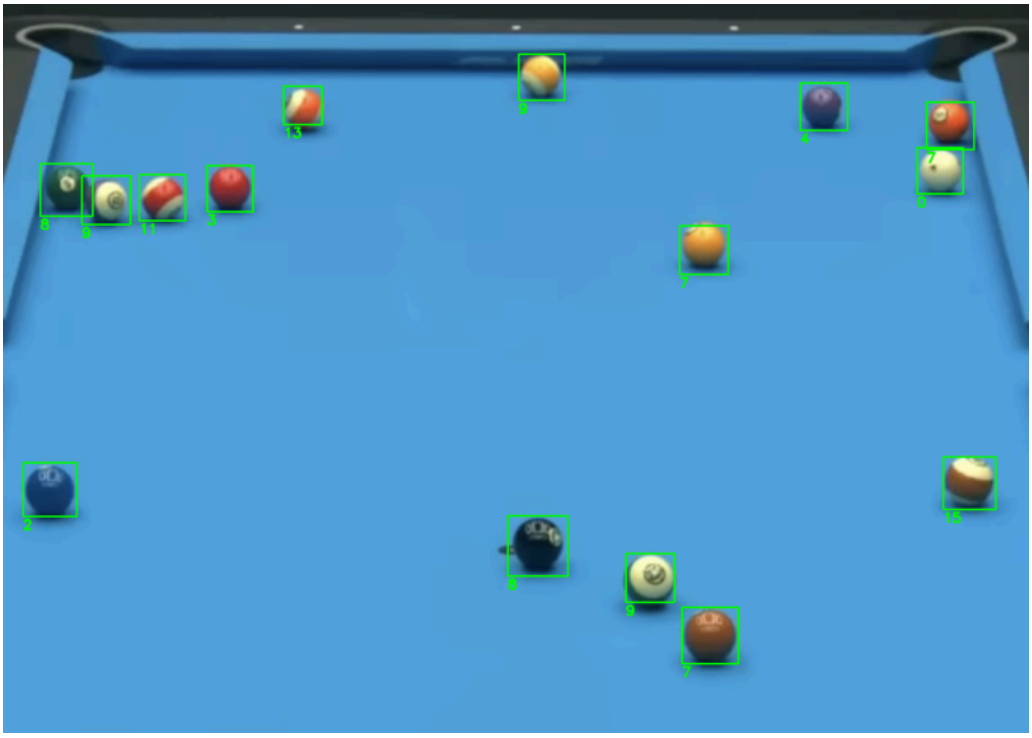


Fig. 3 - Ball Number Identification